

硬判决 RS+BCH 级联码课题

新需求解读与需要对齐的技术问题

陈诗阳

基于 Lagendijk et al. 2026 硬件实现论文

<https://github.com/a-yeyang/efficient-osd-bch-no-ge>

2026-07-23

文档目的

本文档针对老师最新的硬判决级联码任务需求，整理出：

1. Lagendijk 2026 论文的核心技术要点 (§1)
2. 老师新需求的逐条解读 (§2)
3. 相对第一阶段课题的根本变化 (§3)
4. 8 个需要对齐的技术问题 (§4)
5. 建议的执行路线与保守推进策略 (§5)

1 Lagendijk 2026 论文技术要点

1.1 论文定位

论文：Lagendijk et al., “High-throughput Low-latency Hardware Implementation of BCH Decoders,” arxiv 2606.17837v1 (EPFL + Eindhoven Univ. of Technology, 2026 年 6 月)。

这是一篇硬件实现论文，为 BCH 译码器设计 FPGA/ASIC 架构。核心贡献是对比两种译码路径：

路径	算法	支持 t	特点
Conventional	BM (Berlekamp–Massey) + Chien search	任意 t	迭代求 ELP + 穷举根
Direct	直接求 ELP 的根 (闭式代数变换 + LUT)	$t \leq 4$	一次性给出根

1.2 Direct 译码的关键技术

对于 $t = 2, 3, 4$ ，错误定位多项式 (ELP) 的根可以用闭式代数变换直接求出：

- $t = 2$: ELP 是 2 次多项式 $\Lambda(x) = x^2 + S_1x + (S_1^3 + S_3)/S_1$ ，变换成 $A(x) = x^2 + x + k$ 标准形式，用一个大小 $2^m - 1$ 的 LUT 直接查根
- $t = 3$: 变换成 $B(x) = x^3 + x + k$ 或 $C(x) = x^3 + k$ ，用另一个 LUT
- $t = 4$: 4 次多项式，分解为两个 2 次多项式的复合，再查表

硬件收益：Direct 译码把整个 BCH 译码压缩成 3–8 个时钟周期，而 Conventional BM + Chien 需要 $2t + 2$ 时钟周期 + 全 n 位 Chien 搜索。

1.3 论文关键结果 (n=256, Table VI)

Decoder	t	Latency (cyc)	Latency (ns)	FPGA	面积改善
Conv	2	8	16	XCZU48DR	baseline
Direct	2	3	6	XCZU48DR	-62% 时延, -65% LUT, +35% clock speed
Conv	3	8	28	XCZU48DR	baseline
Direct	3	4	8	XCZU48DR	-71% 时延, -75% LUT

Table 1: Conventional vs Direct BCH 译码 (Lagendijk 2026 Table VI)

核心观察

Direct $t = 2$ 译码相对 Conventional 是 $3/8 = 37.5\%$ 时延（相当于 2–3 $\times$ 的硬件层面时延压缩）。这是硬件综合数据，不是软件仿真估算。

2 老师新需求逐条解读

2.1 原文

用类似共享拉格朗日插值理念以及这篇文章 (Lagendijk 2026) 的理念，构建级联码：硬判决 RS + 硬判决 BCH（码长 255/128, $t = 2$ ），比较级联码与硬判 RS 的 BER 性能与时延参量，KPI：级联码时延不高于硬判 RS 的 10%。

2.2 拆解

原文	精确技术含义
共享拉格朗日插值理念	沿用上个课题的思路：内外码共享 α 幂表、denominator products、syndrome 单元
Lagendijk 论文的理念	内码 BCH 采用 Direct root finding + LUT 查表，而非 BM 迭代
硬判决 RS + 硬判决 BCH	外码 RS 用 BM + Chien + Forney；内码 BCH 用 Direct root finding。全硬判，无 LLR
码长 255/128, $t = 2$	两组 config，BCH 都是 $t = 2$ 。 $n = 128$ 严格来说要用 extended BCH
KPI 级联时延 $\leq$ 硬判 RS 时延 + 10%	在时钟周期或 $\mathbb{F}_{2^m}$ ops 口径下，比较级联链路 vs 纯 RS-BM

3 相对第一阶段课题的根本变化

上一阶段（软判决级联）与本次任务（硬判决级联）的对比：

维度	第一阶段（软判决）	本次（硬判决）
内码 BCH	LLOSD (软判决, Lagrange + TEP)	Direct root finding (硬判决, LUT)
外码 RS	BM 硬判 / LCC-BR 软判	只有 BM 硬判
目标基线	软判基线 (LCC-BR)	硬判基线 (RS-BM)
KPI	vs LCC-BR: $\leq 10\%$ (已满足)	vs RS-BM: $\leq 10\%$ (需极致优化)
编码增益	≥ 1 dB	预期 0.5–1 dB (硬判天生偏弱)
主要优化	Lagrange 共享 + TEP 早期终止	BCH $t = 2$ Direct 译码
时延口径	微秒级 ($\mathbb{F}_{2^m}$ ops)	纳秒级 (硬件周期)
可复用比例	—	~70% 基础设施

3.1 可复用的第一阶段成果

- PAM4 modem + AWGN 信道 + bit-LLR（虽然本次只用硬判，但 modem 保持不变）
- RS 编码器 + BM 硬判解码器
- Lagrange 缓存层 (α 幂表 + denominator products)
- 级联链路框架 + Monte Carlo 仿真驱动
- 结果分析和报告脚手架

3.2 本次需要新增的模块

- **BCH $t = 2$ Direct root finding** (核心新工作)
- BCH Conventional BM + Chien 作为对比 baseline
- 硬件时钟周期时延模型（不再只用软件 ops 计数）
- 若需要: $n = 128$ 的 extended BCH 处理

4 需要对齐的 8 个技术问题

按重要性从高到低排列。前 3 个问题会根本决定后续实现路线，建议动手写代码前明确回答。

4.1 Q1. 内码 BCH 的具体译码器

Q1. Direct 还是 Conventional? 还是两者对比?

老师说“硬判决 BCH”，Lagendijk 论文里给出两种硬判决方案。请问：

- 只做 Direct root finding ($t = 2$, 闭式 + LUT)
- 只做 Conventional (BM + Chien)
- 两个都做，用作论文级别的时延消融对比 (推荐)

我建议 (c)：因为论文的核心 claim 就是 Direct vs Conventional 的对比，且我们的 Lagrange 共享理念与 Direct 的 LUT 化最契合（预计算 → 查表）。

4.2 Q2. 时延测量口径

Q2. Clock cycles 还是 F_{2^m} ops?

第一阶段用 F_{2^m} ops 计数，级联相对 RS-BM 慢 $12\times$ ，Case (a) 不通过。本次因为 BCH 内码只用 Direct $t = 2$ (3 cycles)，主要时延优势体现在硬件周期上。请问：

- (a) 只报 clock cycles，用 Lagendijk 论文里的具体数字 (Direct $t=2 = 3$ cyc, RS-BM = ? cyc)
- (b) 同时报 F_{2^m} ops 和 clock cycles (推荐)
- (c) 我们自建 hardware timing model (P 路并行度参数化)

推荐 (b)，能覆盖两种视角，也让老师能横向对比第一阶段结果。

4.3 Q3. 参数配置的具体码率

因为 BCH $t = 2$ 已固定，码率约束需要重新计算：

n	BCH	最匹配的 RS	总码率
$n = 255$	BCH(255, 239) $t = 2$	RS(255, 239) $t = 8$	$0.937 \times 0.937 = 0.879$
$n = 127$	BCH(127, 113) $t = 2$	RS(127, 125) $t = 1$ (太弱)	$0.984 \times 0.890 = 0.876$
$n = 127$ (备选)	BCH(127, 113) $t = 2$	RS(127, 113) $t = 7$	$0.890 \times 0.890 = 0.792$

Table 2: $t = 2$ BCH 下的 RS 参数选择困境

Q3. $n = 127$ 的 RS 参数如何选?

- (a) 严格保持码率 $0.88 \rightarrow$ RS $t = 1$ (纠错能力太弱, 级联几乎无增益)
- (b) 允许码率 $\sim 0.79 \rightarrow$ RS $t = 7$ (强纠错, 但违反 12% 开销约束) (推荐)
- (c) 用 shortened RS 保持精确码率 0.88 (需要额外处理)

4.4 Q4. $n = 128$ 是否放宽为 $n = 127$

Q4. $n = 128$ 用 extended BCH 还是直接 $n = 127$?

Lagendijk 论文的 Direct root finding 只对 **primitive BCH** ($n = 2^m - 1$) 定义。 $n = 128$ 需要额外推广，且 Table VI 里只报告了 $n = 256$ 的具体数字。请问：

- (a) 直接改成 $n = 127$ (更符合论文范式) (推荐)
- (b) 用 extended BCH ($n = 128$)，接受 Direct 译码需要小改
- (c) Shortened BCH (从 $n = 255$ shortened)

4.5 Q5. Lagrange 共享的具体位置

Q5. Direct + BM 共享什么？

Direct BCH 译码用 LUT 查表（不做 Lagrange 插值），RS-BM 用多项式 GF 运算（也不直接用 Lagrange 插值）。第一阶段的 Lagrange 共享（ α 幂表 + denom products）在本课题的效用会大幅降低。

老师说的“共享拉格朗日插值理念”，本次可能落在：

- α 幂表：BCH syndrome ($S_i = r(\alpha^i)$) 与 RS syndrome 使用相同的 α 幂序列 → 硬件可复用
- Syndrome 求值单元：两级共用一个 polynomial evaluator，节省硬件 area
- LUT 表结构：BCH Direct 的 $\{\}_A$ 表可在编码时预填 α 域信息（统一 memory 组织）

需请老师确认，本次的“Lagrange 共享”是否指这类硬件层面的 **module** 复用，而不是软件的 lookup cache（因为软件层面上 Direct 译码基本没有 Lagrange 计算可共享）。

4.6 Q6. 编码增益预期

Q6. 硬判决级联的编码增益是否达标？

硬判 RS + 硬判 BCH 级联，预期编码增益比软判弱。粗略估计：

- 纯 RS-BM $n = 255, t = 8$: FER= 10^{-5} 需 ~ 10.5 dB
- 级联 (RS-BM $t = 8$ + BCH $t = 2$): 预期 0.5–1 dB 编码增益 → FER= 10^{-5} 需 ~ 9.5 –10 dB

请老师明确：

- 只要编码增益 ≥ 0.5 dB 就 OK
- 需要 ≥ 1 dB, 否则考虑增大 BCH t
- 编码增益不是硬 KPI, 重点看时延对比

4.7 Q7. 时延基线的“硬判 RS”具体指哪种

Q7. 硬判 RS 是否使用 Lagendijk 类似的 direct root finding？

Lagendijk 论文只给了 BCH 的 direct 译码。但 RS 也可以用类似的 direct 思路，只是复杂度更高。时延基线（硬判 RS）也用 **direct root finding** 还是 仍用传统 **BM + Chien**？

- 只用 conventional BM + Chien 作基线 (推荐, 与论文一致)
- 也做 direct RS 作对比 (工作量增大)

4.8 Q8. 交付形式

Q8. Python vs Matlab, 是否需要 FPGA 综合?

Lagendijk 论文最强的部分是 **FPGA / ASIC** 综合报告 (Table V/VI)。请问本次交付:

- (a) 只做 Python 仿真 + 抽象时延模型 (基础交付)
- (b) 加上 Matlab port
- (c) 加上 FPGA 综合 (Verilog + Vivado), 产出真实硬件时延数字 (工作量翻倍但最有说服力)

5 建议的执行路线

5.1 保守推进策略 (不等回答的情况下)

如果需要立即开始, 我建议按最保守假设推进, 覆盖 Q1–Q8 所有可能路径:

阶段	任务	时长
Phase 1	实现 BCH $t = 2$ Conventional (BM + Chien)	半天
Phase 2	实现 BCH $t = 2$ Direct root finding (LUT)	1 天
Phase 3	复用现有 RS-BM 与级联链路	半天
Phase 4	硬件时延模型 (clock cycles + $\mathbb{F}_{2^m}$ ops 两种口径)	半天
Phase 5	跑 $n = 255, t = 2$ 完整实验 (Direct + Conv + baseline)	1 天
Phase 6	跑 $n = 127$ (若确认可放宽)	半天
Phase 7	LaTeX 报告 + 图表 + 时延分析	1 天

Table 3: 保守推进路线: 预计 4.5 天完成到 Python 完整仿真

5.2 最小闭环建议

推荐先做的最小闭环

先做 $n = 255 + \text{Direct} + \text{Conv} + \text{RS-BM baseline}$, 跑通并请老师 review 中间结果, 确认路线正确后再横向扩展到 $n = 127$ 和其他配置。
这样即使 Q1–Q8 中若干问题老师后续回答不同, 最小闭环的结果都能作为讨论基础。

6 总结

本文核心

1. Lagendijk 2026 论文的核心是 BCH $t \leq 4$ 的 **Direct root finding** 硬件译码器，对 $t = 2$ 相对 Conventional 有 62% 时延压缩 (3 cyc vs 8 cyc)。
2. 老师本次任务是把 Direct BCH 硬判 + RS-BM 硬判级联，在 PAM4-AWGN 信道下比较与纯 RS-BM 的 BER 和时延，KPI 是级联时延 $\leq$ RS-BM + 10%。
3. 相对第一阶段（软判决），本次只换内码 **BCH** (LLOSD $\rightarrow$ Direct $t=2$)， $\sim 70\%$ 基础设施可复用。
4. 有 8 个关键技术问题需要对齐 (§4)，其中 Q1–Q3 会根本决定路线。
5. 建议先做 $n = 255 + \text{Direct} + \text{Conv} + \text{RS-BM}$ 最小闭环 (预计 3–4 天)，拿到中间结果后再横向扩展。

参考文献

- [1] J. Lagendijk et al., “High-throughput Low-latency Hardware Implementation of BCH Decoders,” arxiv 2606.17837v1, EPFL + Eindhoven Univ. of Technology, Jun 2026.
- [2] L. Yang et al., “Efficient OSD of BCH Codes Without GE,” *IEEE Trans. Inf. Theory*, vol. 71, no. 11, pp. 8294–8311, Nov 2025.
- 第一阶段代码: https://github.com/a-yeyang/efficient-osd-bch-no-ge/tree/main/rs_bch_cascade
- 文档: 2026-07-23 · 陈诗阳